1 Writing a Simple Linux Driver and Recompiling the Kernel

1.1 Setting up the Environment:

```
well@WellVM:~$ uname -r
6.8.0-40-generic
well@WellVM:~$ sudo apt install build-essential linux-headers-$(uname -r)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
build-essential is already the newest version (12.9ubuntu3).
linux-headers-6.8.0-40-generic is already the newest version (6.8.0-40.40~22.04.3).
0 upgraded, 0 newly installed, 0 to remove and 315 not upgraded.
```

1.2 Prepare a simple “Hello World” kernel module and compile:

```
well@WellVM:~/kernel_driver$ make
make -C /lib/modules/6.8.0-40-generic/build M=/home/well/kernel_driver modules
make[1]: Entering directory '/usr/src/linux-headers-6.8.0-40-generic'
warning: the compiler differs from the one used to build the kernel
  The kernel was built by: x86_64-linux-gnu-gcc-12 (Ubuntu 12.3.0-1ubuntu1~22.04) 12.3.0
  You are using:           gcc-12 (Ubuntu 12.3.0-1ubuntu1~22.04.2) 12.3.0
  CC [M]  /home/well/kernel_driver/hello.o
  MODPOST /home/well/kernel_driver/Module.symvers
  CC [M]  /home/well/kernel_driver/hello.mod.o
  LD [M]  /home/well/kernel_driver/hello.ko
  BTF [M] /home/well/kernel_driver/hello.ko
Skipping BTF generation for /home/well/kernel_driver/hello.ko due to unavailability of vmlinux
make[1]: Leaving directory '/usr/src/linux-headers-6.8.0-40-generic'
well@WellVM:~/kernel_driver$ ls
hello.c  hello.ko  hello.mod  hello.mod.c  hello.mod.o  hello.o  Makefile  modules.order  Module.symvers
```

1.3 Loading and unloading the module:

```
well@WellVM:~/kernel_driver$ sudo insmod hello.ko
well@WellVM:~/kernel_driver$ sudo dmesg
[ 6406.146752] hello: loading out-of-tree module taints kernel.
[ 6406.146762] hello: module verification failed: signature and/or required key missing - tainting kernel
[ 6406.147854] Hello, world!
well@WellVM:~/kernel_driver$ sudo rmmod hello
well@WellVM:~/kernel_driver$ sudo dmesg
[ 6406.146752] hello: loading out-of-tree module taints kernel.
[ 6406.146762] hello: module verification failed: signature and/or required key missing - tainting kernel
[ 6406.147854] Hello, world!
[ 6583.151404] Goodbye, world!
```

1.4 Recompiling the kernel to include the module:

- 修改 drivers/misc/Kconfig

```
config HELLO_WORLD
    bool "Hello World Built-in Driver"
    default y
    help
      This is a simple Hello World driver for homework.
```

- 修改 drivers/misc/Makefile

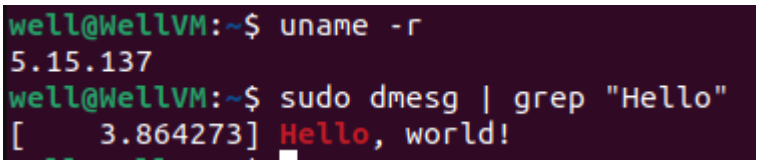
```
obj-$(CONFIG_HELLO_WORLD)       += hello.o
```

- "not a loadable module" 的實作

```
[*] Hello World Built-in Driver
```

1.5 Demonstration:

- Screenshots:



- Short report

1. **環境設定**

首先，使用 uname -r 指令**確認當前系統的核心版本**。接著，透過 sudo apt install build-essential linux-headers-$(uname -r) **安裝編譯所需的開發工具與標頭檔**。為了應對後續的核心重編譯，我也一併安裝了 libncurses-dev、bison、flex 與 libssl-dev 等必要的相依套件。

2. **Hello World 模組實作**

撰寫了一個簡單的核心模組 hello.c。該模組的功能是在初始化時透過 printk 輸出 "Hello, world!"，並在卸載時輸出 "Goodbye, world!"。我同時**編寫了 Makefile 將其編譯為可載入的核心物件檔（.ko）**。 在測試階段，我**使用 insmod 手動載入模組**，並透過 dmesg 確認訊息顯示正確；隨後**使用 rmmod 卸載模組**，再次驗證了卸載訊息的輸出。

3. **核心重編譯與整合**

為了**將此驅動程式整合為核心的內建功能**，而非外部模組，我執行了以下步驟：

1. **下載原始碼**：
    - 下載 Linux Kernel 5.15.137 版本原始碼並解壓縮。
2. **整合程式碼**：
    - 將 hello.c 複製至核心原始碼的 drivers/misc/ 目錄中。
3. **修改配置檔**：
    - 修改 drivers/misc/Kconfig：新增 HELLO_WORLD 設定選項，定義其為 bool 類型並預設為 y。
    - 修改 drivers/misc/Makefile：加入編譯規則，確保該檔案被納入編譯流程。

4. **核心設定**：
    - 為了節省虛擬機的磁碟空間並加速編譯，使用 make localmodconfig 指令，僅保留當前系統所需的驅動程式設定。
    - 使用 scripts/config 指令關閉了 SYSTEM_TRUSTED_KEYS（避免憑證錯誤）以及 DEBUG_INFO_BTF（避免 pahole 錯誤並節省空間）。
    - 進入 make menuconfig，在 "Device Drivers -> Misc devices" 中確認驅動程式選項已勾選為 [*] (Built-in)。
5. **編譯與安裝**：
    - 使用 make -j$(nproc) 進行多核心編譯，過程無誤後，依序執行 make modules_install 與 make install 完成安裝。

4. 結果驗證

重新開機後，進行了以下驗證：

1. 執行 uname -r，確認核心版本已更新為 5.15.137。
2. 執行 dmesg | grep "Hello"，成功在系統開機 Log 中找到 "Hello, world!" 的訊息。

這證明了驅動程式已成功整合進核心，並在系統啟動過程中自動執行，無需手動掛載。

2 Collecting and Processing Temperature Data on MSP430 FR4133 using FRAM

2.1 Setting up the temperature sensor

```c
void initTemperatureSensor() {
    // 配置參考電壓
    // 溫度感測器需要穩定的參考電壓
    PMMCTL0_H = PMMPW_H;              // 解鎖 PMM 暫存器
    PMMCTL2 |= INTREFEN | TSENSOREN; // 啟用內部參考電壓 與 溫度感測器
    __delay_cycles(400);              // 等待參考電壓穩定

    // 配置 ADC 控制暫存器
    // ADCSHT_8: 採樣保持時間.
    //           溫度感測器阻抗較高，需要較長的採樣時間
    // ADCON:    開啟 ADC
    ADCCTL0 = ADCSHT_8 | ADCON;

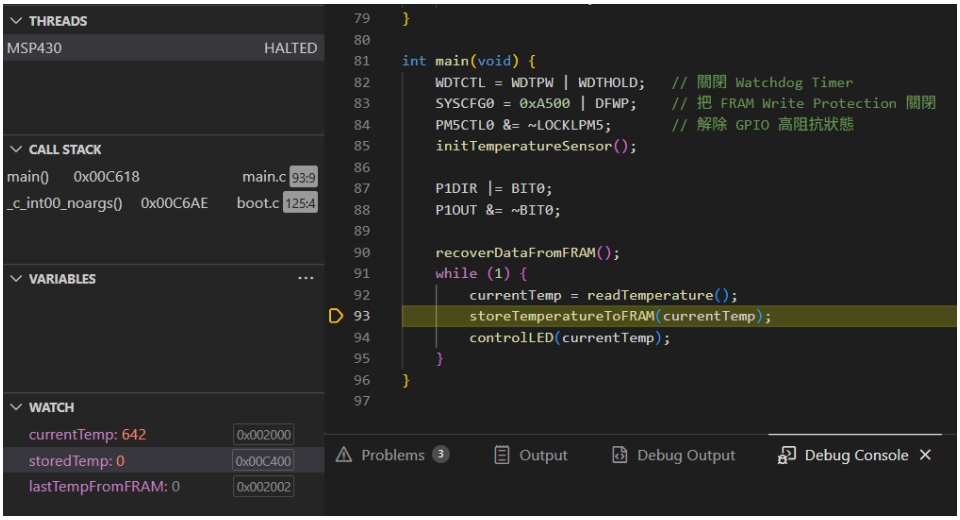
    // ADCSHP:   採樣脈衝模式 (由 SAMPCON 信號源控制)
    // ADCSSEL_1: 時鐘源選擇 ACLK (通常較慢，適合溫度採樣)
    ADCCTL1 = ADCSHP | ADCSSEL_1;

    // ADCRES_1: 10-bit 解析度 (FR4133 是 10-bit ADC)
    ADCCTL2 = ADCRES_1;

    // 配置記憶體控制暫存器 (Memory Control)
    // ADCINCH_12: 在 MSP430FR4133 上，溫度感測器通常連接到 Channel 12
    // ADCSREF_1:  選擇參考電壓 (Vref = Int, Vref- = AVss)
    ADCMCTL0 = ADCINCH_12 | ADCSREF_1;

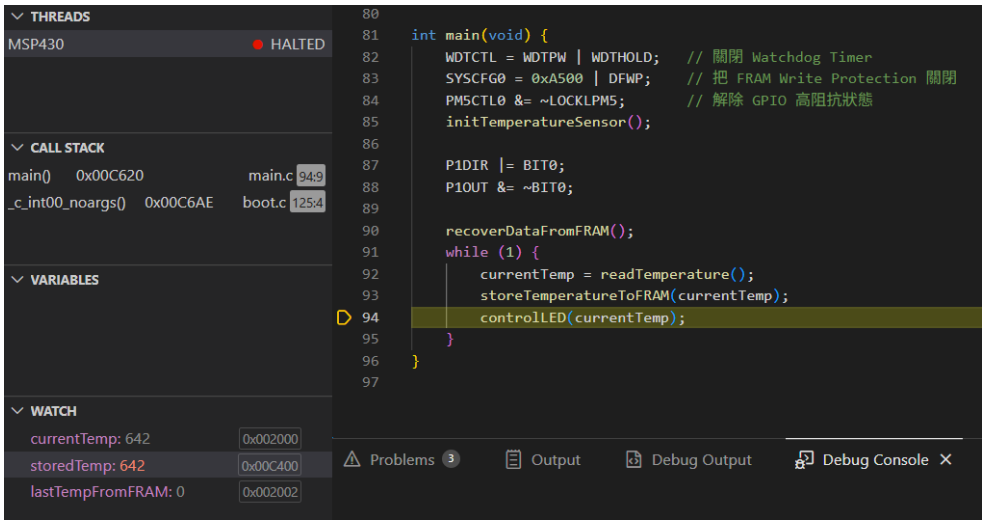
    // 4. 啟用轉換
    ADCCTL0 |= ADCENC;
}
```

```c
unsigned int readTemperature() {
    ADCCTL0 |= ADCSC;               // 開始轉換 (Start Conversion)
    while (ADCCTL1 & ADCBUSY);      // 等待轉換完成 (檢查 BUSY 位元)
    return ADCMEM0;                 // 返回 10-bit 結果
}
```



2.2 Set up FRAM storage for the temperature data

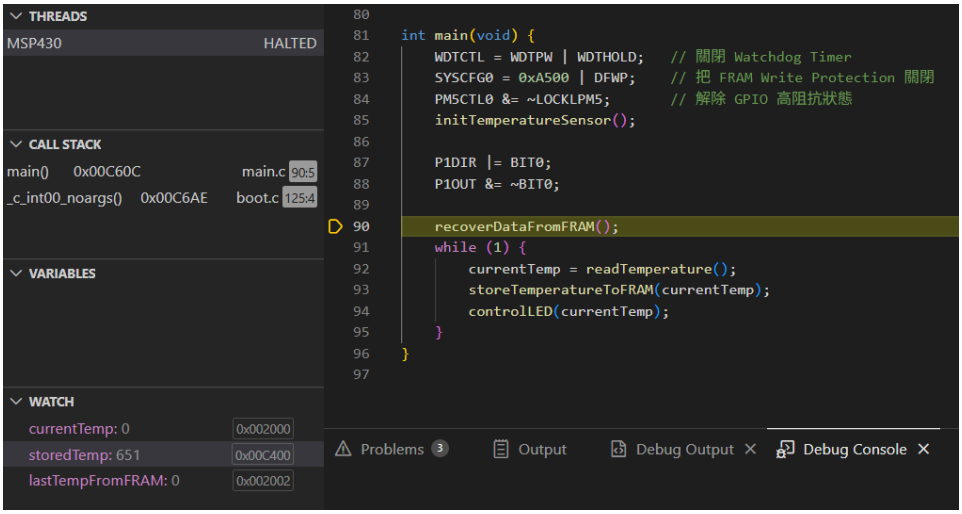
```
void storeTemperatureToFRAM(unsigned int value) {
    storedTemp = value;
    framValid = FRAM_MAGIC;
}
```

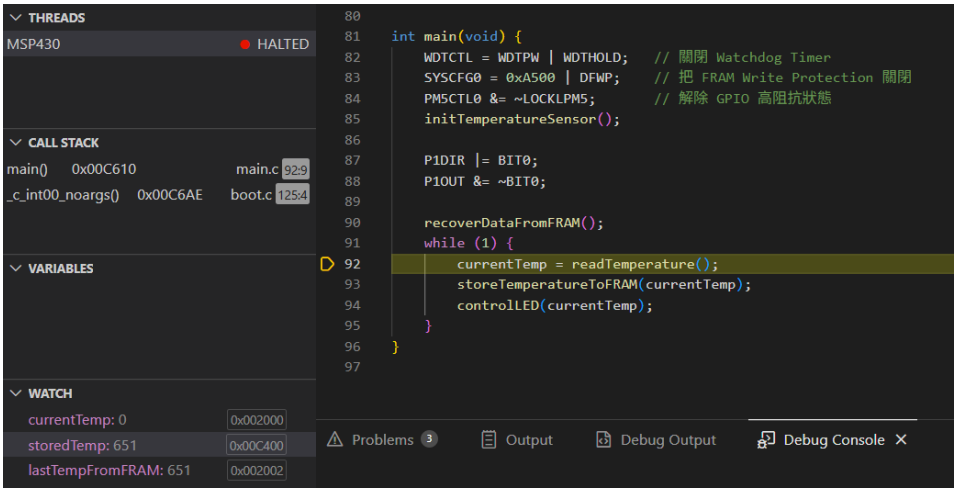


2.3 Power recovery: Ensuring that no data is lost because of power loss

```
void recoverDataFromFRAM() {
    if (framValid == FRAM_MAGIC)
        lastTempFromFRAM = storedTemp;
    else
        lastTempFromFRAM = 0;

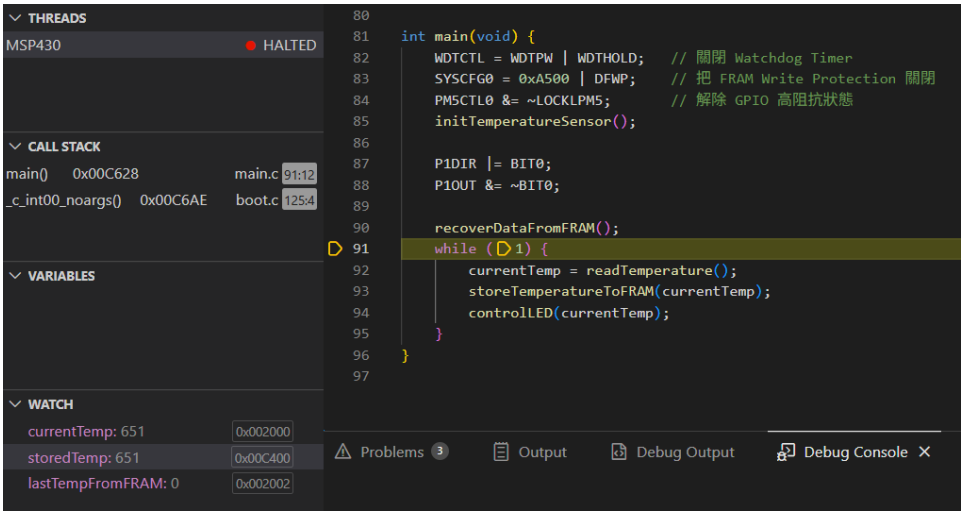
    if (lastTempFromFRAM > 500) {
        // 檢查重啟後的 FRAM 是否有有效資料，有的話閃爍3次
        int i;
        for(i=0; i<3; i++) {
            P1OUT |= BIT0;  // 亮
            P1OUT &= ~BIT0; // 滅
        }
    }
}
```





2.4 If the temperature exceeds a specific threshold, trigger an LED on the board to indicate overheating

```
void controlLED(unsigned int temp) {
    if (temp > TEMP_THRESHOLD)
        P1OUT |= BIT0;
    else
        P1OUT &= ~BIT0;
}
```


THREADS
MSP430
HALTED
CALL STACK
main() 0x00C628 main.c 91:12
_c_int00_noargs() 0x00C6AE boot.c 125:4
VARIABLES
WATCH
currentTemp: 651 0x002000
storedTemp: 651 0x00C400
lastTempFromFRAM: 0 0x002002
80
81 int main(void) {
82 WDTCTL = WDTPW | WDTHOLD; // 關閉 Watchdog Timer
83 SYSCFG0 = 0xA500 | DFWP; // 把 FRAM Write Protection 關閉
84 PM5CTL0 &= ~LOCKLPM5; // 解除 GPIO 高阻抗狀態
85 initTemperatureSensor();
86
87 P1DIR |= BIT0;
88 P1OUT &= ~BIT0;
89
90 recoverDataFromFRAM();
91 while (1) {
92 currentTemp = readTemperature();
93 storeTemperatureToFRAM(currentTemp);
94 controlLED(currentTemp);
95 }
96 }
97
Problems 3
Output
Debug Output
Debug Console

